

บทที่ 4

XML

ในปี ค.ศ. 1991 Tim Bernes-Lee ได้พัฒนา ภาษา HTML ขึ้นเพื่อใช้ในการสร้างเว็บเพจ หลังจากนั้น HTML ก็แพร่หลายและใช้กันอย่างกว้างขวาง จนทำให้มีเว็บเพจจำนวนมากมายนับไม่ถ้วน HTML เป็นมาตรฐานที่ได้รับการปรับปรุงและเสริมต่อมาเป็นลำดับ แต่ HTML ก็ยังเป็นมาตรฐานที่เน้นการนำข้อมูลข่าวสารมาแสดงผลบนจอภาพ ดังนั้น โครงสร้างหลักจึงได้แก่ หัวเรื่อง ชื่อเรื่อง ภาพ ชุดตัวพิมพ์ การแสดงผลด้วยรูปภาพ เสียง และมัลติมีเดีย เพียงเท่านั้นก็ทำให้นักพัฒนาเว็บสามารถออกแบบเว็บเพจได้อย่างน่ามหัศจรรย์ และมีประสิทธิภาพยิ่ง

4.1 ข้อดีของ HTML

ด้วยข้อจำกัดและความต้องการของผู้ใช้ยังมีอีกมาก ทำให้ HTML มีข้อดีบางประการที่จะต้องปรับปรุง ดังต่อไปนี้

- แท็กของ HTML มีจำกัด และไม่อนุญาตให้เราสร้างแท็กขึ้นเองได้ ทำให้การทำงานมีขีดจำกัด แม้ว่าจะมี HTML เวอร์ชันใหม่ๆ ออกมา แต่ก็ยังไม่สามารถตอบสนองกับความต้องการของผู้ใช้ได้ทั้งหมด เพราะรูปแบบแท็กของ HTML มีลักษณะตายตัวว่าแต่ละแท็กเอาไว้ทำอะไร โดยที่แท็กต่างๆ จะถูกกำหนดโดยองค์กร W3C (World Wide Web Consortium) ซึ่งเป็นองค์กรที่มีหน้าที่วางมาตรฐานการทำเอกสารต่างๆบนเว็บ และนอกจากนี้ผู้ผลิต WebBrowser ค่ายใหญ่ๆ เช่น Internet Explorer ของบริษัทไมโครซอฟต์ หรือ Netscape ของบริษัทเน็ตสเคป คอมมิวนิเคชันก็พยายามดึงดูดผู้ใช้เบราว์เซอร์ให้มาใช้ผลิตภัณฑ์ของตนมากขึ้น โดยการเพิ่มเติมแท็กจากแท็กมาตรฐานที่มีอยู่ ทำให้แต่ละเบราว์เซอร์แสดงผลลัพธ์จากไฟล์ HTML เดียวกัน แตกต่างกันไป
- เอกสารที่เขียนด้วย HTML มุ่งเน้นเฉพาะการแสดงผลเป็นหลัก ส่วนการสื่อความหมายของเอกสารนั้นนับว่าน้อยมาก ทำให้การค้นหาข้อมูลที่ต้องการจากเครือข่ายอินเทอร์เน็ต เป็นไปอย่างยากลำบาก คือ ได้ผลลัพธ์ที่ไม่เกี่ยวข้องออกมาเป็นจำนวนมาก
- HTML สามารถให้มุมมองเพียงด้านเดียวแก่ผู้เข้ามาเยี่ยมชม คือเราไม่สามารถทำให้ผู้ใช้หลายๆ คนมองเห็นเพจเดียวกันแตกต่างกันได้ ดังนั้นจึงต้องใช้ ASP และ DHTML (Dynamic HTML) เพื่อให้มองเห็นได้หลายมุมมอง ซึ่งในงานหลายอย่างต้องการคุณสมบัติดังกล่าว โดยเฉพาะอย่างยิ่งการทำธุรกิจบนอินเทอร์เน็ตที่จำเป็นต้องมีการจัดแบ่งกลุ่มลูกค้า เช่น จัดกลุ่มลูกค้าให้แต่ละคนได้มองเห็นเฉพาะในสินค้าส่วนที่เขาสินใจ
- HTML ไม่สนับสนุนหลัก “การนำกลับมาใช้ใหม่ (reuse)” การแก้ไขเอกสารแต่ละครั้งค่อนข้างลำบาก จึงไม่สะดวกกับงานที่มีการเปลี่ยนแปลงข้อมูลบ่อย
- การเปลี่ยนกลับไปมาระหว่าง HTML กับงานอื่นทำได้ยาก เช่น เมื่อเราสร้าง HTML แล้ว จะนำกลับมาใช้งานใหม่ได้ยาก เช่น มีเอกสารเวิร์ดและแปลงเป็น HTML แต่หากจะแปลงกลับทำได้ยากที่จะทำให้ได้เหมือนเดิม

ในปี พ.ศ. 2539 Jon Bosak ได้เสนอแนวคิดต่อ W3C (World Wide Web Consortium) และนำเสนอภาษาใหม่ โดยนำแนวทางบางอย่างมาจาก SGML Standard Generalize Markup Language ซึ่งได้รับการพัฒนามานานแล้ว แต่มีความซับซ้อน ทำให้ใช้งานยาก ข้อเสนอใหม่นี้เรียกว่า XML (Extensible Markup Language) โดยเน้นให้มีส่วนช่วยแสดงส่วนที่เป็นเนื้อหาหรือสาระของข้อมูล

4.2 เป้าหมายหลักของ XML

เป้าหมายที่สำคัญของ XML เน้นที่จะนำไปใช้งานในอินเทอร์เน็ต จึงมีเป้าหมายหลักดังนี้

- เพื่อเป็นมาตรฐานในการทำเอกสารบนเครือข่ายอินเทอร์เน็ตที่ผู้ใช้ทุกคนต่างมีความเข้าใจ และมีข้อตกลงที่ตรงกัน นั่นคือเอกสาร XML จะต้องเป็นรูปแบบที่สามารถใช้ได้ทั่วไปบนอินเทอร์เน็ต
- เพื่อรองรับการแลกเปลี่ยนข้อมูลระหว่างแอปพลิเคชันโดยไม่ต้องขึ้นกับแพลตฟอร์มหรือผู้ผลิตรายใดรายหนึ่ง
- มาตรฐานของ XML จะต้องเข้ากันได้กับมาตรฐานของ SGML ซึ่ง XML นั้นนับเป็นส่วนหนึ่งของ SGML ทำให้องค์กรกำลังใช้ SGML อยู่แล้วก็สามารถนำ XML มาใช้ร่วมด้วยกันได้
- เอกสาร XML จะต้องมีรูปแบบที่ผู้พัฒนาสามารถเขียนโปรแกรมเพื่อนำเอาเอกสารดังกล่าวไปประมวลผลได้โดยง่าย
- การออกแบบเอกสาร XML ควรทำได้ด้วยความรวดเร็ว
- เป็นการแยกส่วนของข้อมูลออกจากการแสดงผลอย่างชัดเจน คือ XML จะไม่บอกเราว่าการแสดงผลที่ได้จะเป็นเช่นไร แต่จะบอกว่าตัวเนื้อข้อมูลจริงๆ นั่นคืออะไร ทำให้การเปลี่ยนแปลงข้อมูลจะไม่ส่งผลกระทบต่อ การแสดงผลเลย ในการกลับกันการแก้ไขส่วนการแสดงผลก็จะมีผลกระทบต่อเนื้อข้อมูลจริงๆ เช่นกัน นอกจากนี้การนำข้อมูลไปแสดงผลนั้น เราจะสามารถเลือกได้ว่าจะใช้อะไรเป็นตัวดึงข้อมูลมาแสดงผล เช่น ใช้ CSS, XSL, VBScript, JavaScript เป็นต้น
- เอกสารที่เขียนด้วย XML จะเป็นเอกสารที่สื่อความหมายได้ดีเพราะภาษา XML จะมีลักษณะของการให้ความหมายแก่ข้อมูลโดยผู้สร้างเอกสารจะเป็นผู้กำหนดนิยามและความหมายของแต่ละอย่าง เองตามข้อตกลงของ W3C ซึ่งจะทำให้การอ่านเอกสารเป็นไปได้ง่ายและเป็นมาตรฐาน เอกสารจะมีความชัดเจน ไม่กำกวม
- XML ออกแบบอย่างพิถีพิถันเน้นความจำเป็น กระทัดรัด เข้าใจง่าย และได้ประโยชน์กว้างขวาง
- XML สนับสนุนการประยุกต์เข้ากับงานต่าง ๆ และสนับสนุนโปรแกรมประยุกต์ต่าง ๆ
- XML เน้นเรื่องการประมวลผลเอกสาร จึงเหมาะกับงานทางด้านการวิเคราะห์เอกสาร การผลิตเอกสาร การแลกเปลี่ยน และการแสดงผล

- การเขียน XML ทำได้ตั้งแต่การใช้ Text editor ทั่ว ๆ ไป และไม่ต้องการเครื่องมือที่ซับซ้อน ใด ๆ ไรก็ดี ย่อมต้องมีผู้เขียน XML editor ให้ใช้งานได้ง่ายขึ้น
- XML เป็นมาตรฐานที่กำหนดแล้วใช้งานได้ทันที โดยที่ Browser และอุปกรณ์ต่าง ๆ พร้อม ใช้งานร่วมกัน

4.3 การประยุกต์ใช้ XML

ในปัจจุบันมีการจัดการเอกสารจำนวนมากการบนเครือข่าย เช่น การสร้างห้องสมุดดิจิทัล การ แลกเปลี่ยนข้อมูลข่าวสารระหว่างกัน การประยุกต์ใช้ XML จึงทำได้อย่างกว้างขวาง ตัวอย่างเช่น

- XML สามารถใช้ได้หลากหลายภาษา และผสมกันได้หลากหลายภาษา เนื่องจาก XML สนับสนุน UNICODE
- การพัฒนา XML Processor ทำให้สามารถดึงเอกสาร XML มาใช้งานได้ง่าย และใช้ร่วมกับ โปรแกรมประยุกต์อื่นได้ง่าย เช่น โปรแกรม DB2, Oracle, SAP เป็นต้น
- XML ช่วยทำให้เกิดการรับส่งข้อมูลแบบ EDI โดยทำให้แนวทางการเชื่อมโยงและสร้าง ความเป็นเอกสารหรือมาตรฐานระหว่างองค์กร
- XML มีสภาพช่วยในการขนส่งข้อมูลไปยังปลายทางเพื่อให้แปลความหมายและใช้งานได้ อย่างเต็มประสิทธิภาพ
- มีการสร้างการประยุกต์ และนำเสนอผลลัพธ์ไปใช้งานจาก XML ได้มาก
- การประยุกต์การดำเนินกิจกรรมบนเครือข่ายมีมาก เช่น e-Business, EDI, e-Commerce การ จัดการ Supply chain, Demand chain management การดำเนินการแบบ intranet และ web base application

4.4 XML จะแทนที่ HTML หรือไม่

แม้ว่าแต่เดิม XML จะถูกสร้างขึ้นมาเพื่อแก้ไขข้อบกพร่องของภาษา HTML แต่ XML ก็ไม่ใช่สิ่ง ที่จะมาแทน HTML อย่างที่หลายๆ คนเข้าใจผิด เพราะ XML เป็นเพียงส่วนเนื้อหาที่แท้จริงของเอกสารเท่านั้น ส่วนในเรื่องการแสดงผลนั้น ถ้าฟัง XML อย่างเดียวไม่สามารถบอกได้ว่าจะแสดงผลเป็นเช่นไร (ซึ่งก็คือ XML ไม่สามารถแสดงผลได้ด้วยตัวของมันเอง) ดังนั้น HTML ก็จะต้องอยู่ต่อไปอย่างแน่นอน อย่างน้อยก็อีกระยะหนึ่ง ในขณะที่เดียวกัน HTML ก็จะมีพัฒนาการและการปรับเปลี่ยนโดยดึงเอาข้อดีของทั้ง XML และ HTML มารวมกันกลายเป็น XHTML ขึ้นมาใช้งานแทน

4.5 โครงสร้าง XML

XML ถูกวางให้มีบทบาทสำคัญในฐานะที่เป็นตัวกลางของการสื่อสารระหว่างโปรแกรม เพราะเราสามารถสื่อสารระหว่างระบบงานที่แตกต่างกันโดยใช้ความสามารถของ XML SOAP และ HTTP ซึ่งจะไม่มีปัญหาเกี่ยวกับ firewall เพราะ HTTP ใช้ port 80 ในการติดต่อสื่อสาร นี่คือตัวอย่างของไฟล์ XML

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="show_book.xsl"?>
<!DOCTYPE catalog SYSTEM "catalog.dtd">
<!--catalog last updated 2000-11-01-->
<catalog xmlns="http://www.example.com/catalog/">
  <book id="bk101">
    <author>Abercrombie, Kim</author>
    <title>XML Developer's <#x47;uide</title>
    <genre>Computer</genre>
    <price>44.95</price>
    <publish_date>2000-10-01</publish_date>
    <description><![CDATA[An in-depth look at
creating applications with XML, using <, >, ]]> and
&amp; .</description>
  </book>
  <book id="bk109">
    <author>Kress, Peter</title>
    <title>Paradox Lost</title>
    <genre>Science Fiction</genre>
    <price>6.95</price>
    <publish_date>2000-11-02</publish_date>
    <description>After an inadvertant trip through
a Heisenberg Uncertainty Device, James Salway discovers the
problems of being quantum.</description>
  </book>
</catalog>
```

รูปที่ 4-1 ตัวอย่างไฟล์ XML

จากภาพข้างบนจะเห็นว่า XML จะประกอบด้วย 2 ส่วนคือ

1. ส่วนที่เป็น Prolog โดยบรรทัดแรกจะเหมือนการประกาศทั่วไปว่า XML ตัวนี้เป็น version 1 ซึ่งเป็น version ปัจจุบัน และถ้าเราต้องการให้สามารถแสดงภาษาไทยได้ ก็จะต้องเปลี่ยน encoding เป็น encoding="windows-874" ถัดบรรทัดลงมาจะเป็นการ link file XSL เข้ามาในไฟล์ XML เพื่อจัดรูปแบบการแสดงผล (เหมือนกับที่เรา link ไฟล์ CSS เข้ามาช่วยจัดรูป HTML ของเรา) บรรทัดต่อมา เป็นการกำหนดรูปแบบของไฟล์ XML ว่าถูกต้องตามที่กำหนดใน DTD หรือไม่ ที่ต้องมีการทำเช่นนี้ ก็เพราะว่าเราสามารถเพิ่มเติมส่วนต่างๆ เข้าใน XML ของเราได้ตามชอบใจ ซึ่งถ้าเราใช้ของเราคนเดียวก็จะไม่มีปัญหาอะไร แต่ถ้าเราต้องมีการแลกเปลี่ยนข้อมูลกัน ซึ่งก็จะใช้รูปแบบเดียวกัน โดย DTD จะเป็นตัวกำหนดรูปแบบความถูกต้องของไฟล์ XML (Well Formed)
2. ส่วนที่เป็น Document Elements คือส่วนที่เป็นข้อมูล โดยจะแบ่งข้อมูลเป็นส่วนๆ เป็น element โดย element บนสุดจะเรียก Root tag หรือ โหนดแม่ แล้วจึงค่อยแตกเป็น โหนดลูก จากตัวอย่าง

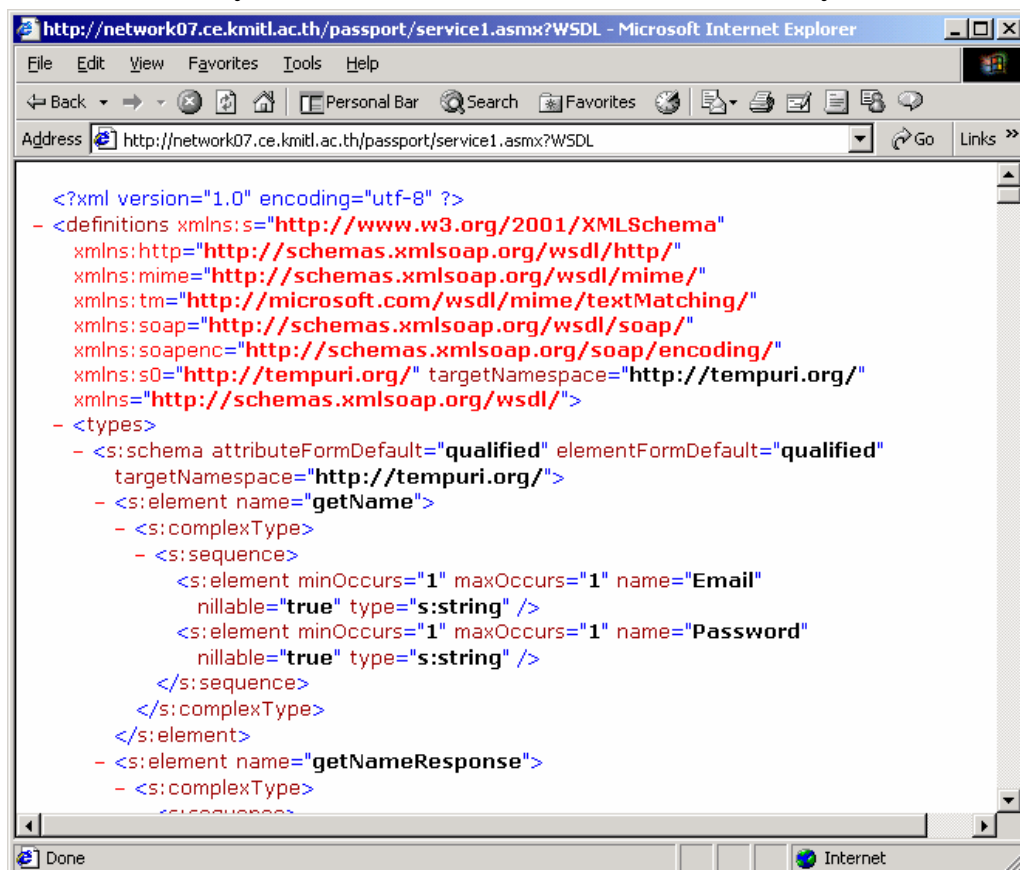
ข้างบนนี้ เราจะมี catalog เป็น Root node และมีโหนดลูกเป็น book แต่ละเล่ม และในโหนดลูกก็สามารถจะมี element ย่อยๆ ได้อีกตามความต้องการ และในแต่ละ element ก็อาจจะมี attribute ได้เหมือนกัน โดยจะอยู่ในรูป " " จากตัวอย่างข้างบน เราใช้ attribute id เป็นตัวแยก book แต่ละเล่ม

4.6 ข้อควรระวังในการเขียน XML

ในการสร้างเอกสาร XML มีข้อควรระวัง ดังต่อไปนี้

- เมื่อเปิดแท็กแล้วจะต้องปิดแท็กด้วยเสมอ เช่น <p> ย่อหน้าที่ 1 </p>
- ในแท็กของ XML นั้น อักษรตัวใหญ่และเล็กนั้นมีความแตกต่างกัน (Case Sensitive) เช่น <Ticket>จะไม่เหมือนกับ <ticket>
- การจัดการเรื่องตำแหน่งแท็กของ XML ต้องเป็นไปตามลำดับ คือจำเป็นต้องเปิดและปิดเป็น วงๆ ไป เช่น <i>ข้อความนี้จะเป็นตัวหนาและเอียง </i>
- จำเป็นต้องมี Root Tag โดยมีแท็กลูกอื่นๆ อยู่ในวงของแท็กแม่ และถ้าแท็กลูกมีแท็กหลาน แท็กหลานก็ต้องอยู่เป็นวงๆ ภายในกรอบของแท็กลูก จะไม่มีการฉีกวงได้เหมือนใน HTML
- attribute ของ XML ต้องอยู่ ภายในกรอบของเครื่องหมายฟันหนู (") เสมอ

เมื่อเราพิมพ์ Browse ดูไฟล์นี้ในโปรแกรม Internet Explorer จะได้ผลลัพธ์ ดังรูป



รูปที่ 4-2 ผลลัพธ์จาก Browse ดูในโปรแกรม Internet Explorer

4.7 วิเคราะห์โครงสร้างของเอกสารแบบ DOM และ SAX

ในการพัฒนาแอปพลิเคชันโดยมีการนำ XML ไปใช้งานนั้น สิ่งหนึ่งที่ผู้พัฒนาจำเป็นต้องทำการพาร์ส (parse) เอกสาร XML เพราะเนื่องจากว่าโปรแกรมจะมองเห็นเอกสารเป็นเพียงอักขระธรรมดาๆ เท่านั้น เราจึงต้องมีการเขียนโปรแกรมเพื่อให้แอปพลิเคชันสามารถเข้าใจได้ว่าเอกสาร XML ของเรามีการจัดโครงสร้างเป็นอย่างไร โดยสิ่งที่เราใช้ในการพาร์สเอกสารเราจะเรียกว่าพาร์สเซอร์ (Parser) จากตัวอย่างที่ผ่านมาแล้ว เราสามารถใช้โปรแกรม Internet Explorer มาใช้กับเอกสาร XML ของเราได้ เนื่องจากโปรแกรมนี้นี้ได้มีการฝังโปรแกรมพาร์สเซอร์เอาไว้ในตัวอยู่แล้ว เรียกว่า Microsoft XML Parser เราจึงไม่ต้องทำการกำหนดการอ้างอิงพาร์สเซอร์เหมือนเวลาที่เราเขียนโปรแกรมทั่วไป ในปัจจุบันมีมาตรฐานหรือวิธีการหลักๆ อยู่ 2 อย่างในการพาร์สเอกสาร XML ได้แก่ DOM (Document Object Model) และ SAX (Simple API for XML) ซึ่งทั้ง 2 วิธีนี้ต่างก็มีจุดเด่นและจุดด้อยอยู่ในตัวเอง การใช้งานจึงขึ้นกับความต้องการในการใช้งานของแอปพลิเคชันที่ใช้เอกสาร XML นั้นว่ามีความเหมาะสมกับวิธีการใดมากที่สุด

4.7.1 การพาร์สเอกสาร XML โดยใช้ DOM (Document Object Model)

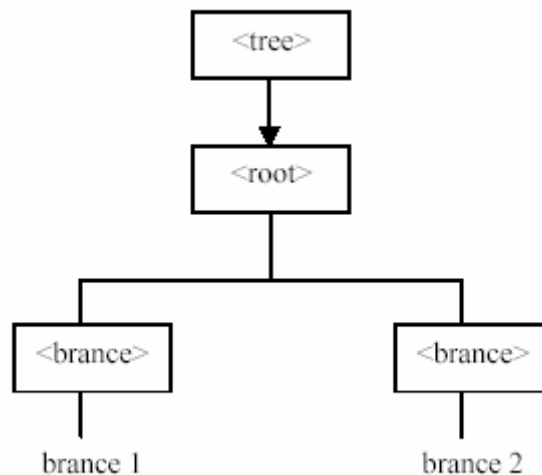
XML DOM เป็น API (Application Programming Interface) สำหรับเอกสาร XML ซึ่งจะกำหนดว่าเอกสาร XML จะมีการเข้าถึง การโยกย้ายและการจัดการต่างๆ ได้อย่างไรบ้าง ด้วยวิธีการแบบ DOM ผู้พัฒนาจะสามารถเข้าไปในโครงสร้างระดับต่างๆ ของเอกสาร XML ได้ รวมทั้งยังสามารถเพิ่ม ลบ และแก้ไขอีลิเมนต์ต่างๆ ของเอกสารได้อีกด้วย

DOM เป็นมาตรฐานของ W3C โดยมีเป้าหมายคือ ต้องสามารถสร้างอินเตอร์เฟซที่เป็นมาตรฐาน และจะต้องใช้ได้กับแอปพลิเคชันได้หลากหลายการทำงานของ DOM จะเริ่มต้นเมื่อโปรแกรมหรือแอปพลิเคชันเรียกพาร์สเซอร์เพื่อทำการโหลดเอกสาร XML เข้าสู่หน่วยความจำของคอมพิวเตอร์ เมื่อเอกสารถูกโหลดเสร็จเรียบร้อยแล้ว DOM ก็จะทำให้โปรแกรมสามารถดึงข้อมูลมาใช้งานหรือดำเนินการต่างๆ กับข้อมูลได้ DOM จะมีมุมมองไปยังเอกสาร XML เป็นโครงสร้างแบบต้นไม้ (Tree) โดยระดับบนสุดของทรีเราจะเรียกว่า documentElement ซึ่งอีลิเมนต์ดังกล่าวจะแตกสาขาออกไปเป็นอีลิเมนต์ย่อยๆ ตั้งแต่ 1 อีลิเมนต์ขึ้นไป ซึ่งเราจะเรียกว่า โหนดลูก (childNodes) จากเอกสาร XML ที่มีรายละเอียดดังด้านล่างนี้

```
<tree>
  <root>
    <brance> brance 1 </brance>
    <brance> brance 2 </brance>
  </root>
</tree>
```

รูปที่ 4-3 ตัวอย่างไฟล์ XML

DOM จะมองเอกสารข้างต้นเป็น ดังนี้คือ



รูปที่ 4-4 DOM Tree

จุดเด่นของ DOM

- โปรแกรมจะพาร์สเอกสารเพียงครั้งเดียว เมื่อ DOM ได้สร้างทรีขึ้นมาแล้ว ทรีนั้นก็จะอยู่ในหน่วยความจำตลอดจนกว่าเราจะสั่งยกเลิก และในขณะที่ทรีอยู่ในหน่วยความจำ เราก็สามารถเข้าถึงข้อมูลที่อยู่ภายในทรีได้อย่างรวดเร็ว
- ข้อมูลที่สร้างเป็นทรีจะทำให้ข้อมูลเกิดความสัมพันธ์ ทำให้การเรียกใช้ทำได้ง่าย

จุดด้อยของ DOM

- ในกรณีที่เอกสารมีขนาดใหญ่และซับซ้อน การสร้างทรีจะทำให้ช้า และกินหน่วยความจำมาก
- ผู้พัฒนาโปรแกรมจำเป็นต้องเข้าใจถึงโครงสร้างโดยรวมของเอกสารเป็นอย่างดี จึงจะสามารถเขียนโปรแกรมโดยใช้ DOM ได้ กล่าวคือในส่วนของการเขียนโค้ดด้วยวิธีการของ DOM นั้น จะค่อนข้างยาวและยุ่งยากอยู่พอสมควร

4.7.2 การพาร์สเอกสาร XML โดยใช้ SAX (Simple API for XML)

SAX เป็น API ที่มีลักษณะการทำงานที่ต่อเนื่องเมื่อมีเหตุการณ์ (เช่นการร้องขอข้อมูล) ใดๆ เกิดขึ้น หรือเรียกโมเดลการทำงานแบบนี้ว่า event-based model กล่าวคือ SAX จะไม่มีการสร้างภาพรวมของเอกสารไว้ก่อนเลยว่า เอกสารนั้นมีโครงสร้างเป็นอย่างไร เมื่อแอปพลิเคชันมีการร้องขอมาจึงจะรายงานหรือทำงานตามเหตุการณ์ของการทำพาร์สซิง (parsing) ไปยังแอปพลิเคชัน

หลักการทำงานของ SAX จะมองเพียงจุดเริ่มต้นและจุดสิ้นสุดของเอกสารและของอีลีเมนต์เท่านั้น และจะสนใจเฉพาะสิ่งที่ต้องการเท่านั้น ทำให้ SAX เข้าถึงข้อมูลได้ง่ายกว่า DOM รวมถึงการเขียนโปรแกรมก็ทำได้ง่ายกว่า เพราะมองข้อมูลในลักษณะเส้นตรงและสามารถพาร์สเอกสารที่มีขนาดใหญ่ มากกว่าจำนวนหน่วยความจำที่เรียกได้ ดังนั้นโปรแกรมระบบฐานข้อมูลใหญ่ๆ หลายบริษัทจึงนิยมใช้วิธีนี้

จากเอกสาร XML ที่มีรายละเอียดดังด้านล่างนี้ SAX จะมองเอกสารข้างต้นไปที่ละบรรทัด ในลักษณะเหตุการณ์เชิงเส้น ดังนี้

```

<tree>
  <root>
    <brance> brance 1 </brance>
    <brance> brance 2 </brance>
  </root>
</tree>

```

รูปที่ 4-5 ตัวอย่างไฟล์ XML

SAX จะมองเอกสารข้างต้นไปที่ละบรรทัด ในลักษณะเหตุการณ์เชิงเส้น ดังนี้

start document	↓	พาร์สเอกสาร
start element: doc		ตั้งแต่บรรทัดแรก
start element: element1		จนถึงบรรทัดสุดท้าย
characters: Value 1		แบบเส้นตรง
end element: element1		
start element: element2		
characters: Value 2		
end element: element2		
end element: doc		
end document		

รูปที่ 4-6 SAX Event

ตัวอย่างการพาร์สแบบ SAX ข้างต้นเป็นการพาร์สทั้งเอกสาร ในทางปฏิบัติแล้วเรามักไม่ได้มีการพาร์สเอกสารทั้งหมด เมื่อเราได้ข้อมูลที่เราต้องการแล้วเราก็จะหยุด วิธีการพาร์สแบบ SAX จึงทำงานได้อย่างรวดเร็วและเขียนโปรแกรมได้ง่ายมาก

ตัวอย่างการพาร์สแบบ SAX ข้างต้นเป็นการพาร์สทั้งเอกสาร ในทางปฏิบัติแล้วเรามักไม่ได้มีการพาร์สเอกสารทั้งหมด เมื่อเราได้ข้อมูลที่เราต้องการแล้วเราก็จะหยุด วิธีการพาร์สแบบ SAX จึงทำงานได้อย่างรวดเร็วและเขียนโปรแกรมได้ง่ายมาก

จุดเด่นของ SAX

- เข้าถึงข้อมูลได้ง่ายและรวดเร็ว
- เขียนโปรแกรมได้ง่าย เนื่องจากการเข้าถึงข้อมูลเป็นลักษณะเส้นตรง

- ใช้หน่วยความจำน้อย สามารถเข้าถึงข้อมูลหรือเอกสารที่มีขนาดใหญ่กว่าหน่วยความจำที่มีของเครื่องได้

จุดด้อยของ SAX

- ไม่สามารถบอกความสัมพันธ์ของข้อมูลได้
- เป็นวิธีการที่ไม่ค่อยมีประสิทธิภาพหากเป็นการเข้าถึงข้อมูลเดิมบ่อยๆ